HIVEMIND

AI Layer — Complete Developer Specification

Swarm Architecture · Hybrid Memory · Graph RAG · Agent Personalities
Developer Handoff Document · Sprint-by-Sprint Build Plan

Assigned developer: 1 full-stack engineer

`Stack: Python 3.12 · Redis · Qdrant · Neo4j · PostgreSQL`

Timeline: 10 sprints · ~14 weeks to full agentic AI layer

1. Design Philosophy

The central principle of the HIVEMIND AI layer: the LLM is a reasoning engine, not a database, not an oracle, and not a personality simulator bolted onto a retrieval system as an afterthought. Every architectural decision in this document flows from that single constraint.

The golden rule

An agent that cannot explain WHY it made a call — citing specific memory, specific retrieved context, and a falsifiable thesis — is not allowed to make that call. No explanation = no trade recommendation.

1.1 What the LLM does NOT own

- Market data — owned by TimescaleDB
- Factor scores — owned by the quant engine
- News history — owned by the vector store
- Trade thesis history — owned by graph + vector store
- Agent memory — owned by the multi-level memory system
- Retrieval — owned by the hybrid BM25 + vector + graph pipeline
- Verification — owned by the critic agent, not the generator

1.2 What the LLM DOES own

- Synthesis: combining retrieved context into a coherent argument
- Planning: deciding what to look for and in what order
- Reasoning: multi-hop inference over structured retrieved facts
- Opinion: applying trained priors and personality biases to produce a genuine stance
- Communication: writing the trade thesis in a specific voice and style

Architecture shift

This system follows the 2026 agentic RAG pattern: retrieve → reason → identify gaps → re-retrieve → synthesise → verify. A single retrieval pass is never sufficient for a trade decision. Every agent performs at least two retrieval rounds.

1.3 Why agent personality matters

Markets are disagreement engines. A committee of agents that all think the same way is worse than one good analyst. HIVEMIND deliberately engineers cognitive diversity into its swarm: each agent has a trained-in personality, explicit biases, and known blind spots. The orchestrator's job is to run a structured adversarial debate — not a consensus poll — and produce a decision that survives the disagreement.

An agent's personality is not cosmetic. It determines: what it retrieves first, how it weights conflicting evidence, what it treats as a red flag vs. noise, and how confidently it holds a view under pressure from other agents. These differences produce genuinely different outputs, which is the point.

2. Memory Architecture

HIVEMIND implements a five-tier memory hierarchy. Each tier has a different latency, persistence duration, and access pattern. No tier is optional — they solve different problems. The LLM never reads from storage directly; it always goes through the Memory Manager, which decides which tier(s) to query and how to compress the results before injecting them into context.

Tier	Name	Storage	TTL	What it stores	Access latency
T1	Working memory	In-process Python dict	Current run only	Active agent state, current task inputs, intermediate reasoning steps	< 1ms
T2	Episodic memory	Redis 7 (sorted sets + hashes)	7 days rolling	Recent trade decisions, recent news summaries, recent agent outputs per ticker	1–5ms
T3	Semantic memory	Qdrant (vector embeddings)	Permanent	All trade theses, research notes, post-mortem findings, extracted market knowledge	10–30ms
T4	Procedural memory	PostgreSQL (agent_procedures table)	Permanent, versioned	Learned workflows: "when I see X+Y in pharma, do Z first", weight adjustments	5–15ms
T5	Knowledge graph	Neo4j (free Docker)	Permanent	Entity relationships: stock↔sector↔catalyst↔outcome↔agent_call chains	20–80ms

2.1 Working Memory (T1)

Working memory is the agent's scratchpad for a single run. It is a Python dict passed through the agent execution chain. It is never persisted. It holds: the current ticker and sector, the raw retrieved context from T2/T3/T5, the outputs of parallel sub-agents, intermediate reasoning steps, and the final structured output before it is written to T2/T3.

```
# T1 structure — passed into every agent call
working_memory = {
    "ticker":         "CDSL",
    "sector":         "FINSERV",
    "run_id":         "2024-01-15T21:00:00Z",
    "regime":         "RISK_ON",
    "factor_scores": { ... },          # from Gate 4
    "t2_episodes":   [ ... ],          # last 5 decisions for this ticker
    "t3_similar":    [ ... ],          # top 5 semantically similar past theses
    "t5_graph":      { ... },          # graph neighbourhood of this ticker
    "agent_outputs": {},               # populated as sub-agents complete
    "retrieval_log": [],               # every retrieval call logged here
    "reasoning_steps": [],             # chain-of-thought scratchpad
}
```

2.2 Episodic Memory (T2) — Redis

Episodic memory stores the recent operational history of the swarm. It answers: "What did we think about this stock last week? What news triggered our last call? What did the critic agent flag?" Every

completed agent run writes a compressed episode to Redis. Episodes expire after 7 days by default, 30 days for stocks with open positions.

```
# Redis key schema
episode:{ticker}:{run_date}     → Hash {
    decision, confidence, entry, stop, target,
    news_sentiment, catalyst_tags,
    agent_agreement_score,      # 0-1: how much agents agreed
    critic_flags,               # list of issues critic raised
    thesis_summary,             # 150-word compressed thesis
    vector_id,                  # pointer to T3 full embedding
    graph_node_id               # pointer to T5 entity
}

ticker:recent_decisions:{ticker} → Sorted set, score=timestamp
sector:recent_decisions:{sector} → Sorted set, score=timestamp
agent:mistake_log:{agent_name}   → List of last 20 flagged errors
system:daily_regime              → String (RISK_ON/CAUTIOUS/RISK_OFF)
```

The episodic memory is the primary mechanism by which agents learn from recent mistakes. Before every run, the Memory Manager fetches the agent's personal mistake log from Redis and injects it into the system prompt as "recent errors to avoid." This is how the system avoids repeating the same wrong call two nights in a row.

2.3 Semantic Memory (T3) — Qdrant

Semantic memory is the long-term knowledge base. Every agent output is embedded and stored permanently. The key design decision: each agent writes to its own Qdrant collection, and the Memory Manager performs cross-collection retrieval when the orchestrator needs a multi-perspective view.

Qdrant collection	Agent that writes	Embedding model	Payload fields	Retrieval trigger
trade_theses	Orchestrator	BAAI/bge-m3 (1024d)	ticker, sector, date, decision, confidence, pnl_outcome (updated post-close)	New ticker: find 5 most similar past setups regardless of time
news_episodes	News analyst	BAAI/bge-m3 (1024d)	ticker, headline, sentiment, catalyst_type, date	Find similar news events for same ticker or sector in last 2 years
research_notes	Researcher agent	BAAI/bge-m3 (1024d)	ticker, fundamental_snapshot, moat_assessment, risk_flags, date	Pull last 3 research notes for ticker before new fundamental research
quant_signals	Quant analyst	paraphrase-MiniLM (384d)	ticker, factor_snapshot, score_vector, timeframe, date	Find historically similar factor configurations that resolved bullishly

Qdrant collection	Agent that writes	Embedding model	Payload fields	Retrieval trigger
post_mortems	Feedback agent	BAAI/bge-m3 (1024d)	ticker, sector, exit_reason, pnl_pct, what_worked, what_failed, date	Injected into every agent run: "here is what failed in similar setups"
market_knowledge	Macro analyst	BAAI/bge-m3 (1024d)	regime, vix_level, sector_rotation, date, macro_thesis	Fetch regime context at start of every nightly run

2.4 Procedural Memory (T4) — PostgreSQL

Procedural memory stores learned workflows — not what the agent knows, but how it has learned to act. It is updated by the Feedback Agent monthly based on P&L attribution analysis. Agents query their own procedural memory at startup to load their current decision weights and sector-specific heuristics.

```
CREATE TABLE agent_procedures (
    agent_name          VARCHAR(30),
    sector              VARCHAR(60),
    procedure_key       VARCHAR(100),     -- e.g. "delivery_spike_weight_pharma"
    procedure_value     JSONB,            -- current learned value
    confidence          NUMERIC,          -- 0-1: how much evidence backs this
    sample_size         INTEGER,          -- number of trades this is based on
    last_updated        TIMESTAMPTZ,
    version             INTEGER,
    PRIMARY KEY (agent_name, sector, procedure_key, version)
);
```

```
-- Example rows
-- ("quant_analyst", "PHARMA", "delivery_spike_weight", 0.35, 0.72, 47, ...)
-- ("news_analyst",  "PHARMA", "usfda_alert_severity",  "critical", 0.91, 23, ...)
-- ("orchestrator",  "IT",     "override_threshold",    0.85, 0.68, 31, ...)
```

2.5 Knowledge Graph (T5) — Neo4j

The knowledge graph is the most powerful and most underused component in most RAG systems. HIVEMIND uses Neo4j (free Docker, Community Edition) to build a persistent graph of entities and relationships across all market activity. Graph retrieval solves multi-hop reasoning problems that vector search fundamentally cannot address.

Node types

Node label	Properties	Example
Stock	symbol, sector, market_cap, is_fo_eligible	(:Stock {symbol:"CDSL", sector:"FINSERV"})
Sector	name, macro_sensitivity, benchmark_index	(:Sector {name:"FINSERV", macro_sensitivity:"HIGH"})

Node label	Properties	Example
Catalyst	type, description, date, severity	(:Catalyst {type:"EARNINGS_BEAT", severity:0.8, date:"2024-01-15"})
TradeDecision	run_id, decision, confidence, pnl_outcome	(:TradeDecision {decision:"BUY", confidence:0.82, pnl_outcome:0.031})
NewsEvent	headline, sentiment, source, date	(:NewsEvent {sentiment:0.7, source:"MONEYCONTROL"})
MacroRegime	signal, vix_pct, fii_net, date	(:MacroRegime {signal:"RISK_ON", vix_pct:0.42})
Agent	name, model, personality_type	(:Agent {name:"QUANTRA", model:"nemotron-70b"})
Mistake	error_type, description, date, corrected	(:Mistake {error_type:"MISSED_RED_FLAG", corrected:false})

Relationship types (edges)

Relationship	From → To	Properties	Graph query use case
BELONGS_TO	Stock → Sector	weight (0-1)	Find all stocks in a sector that recently had delivery spikes
TRIGGERED	Catalyst → Stock	date, impact_score	When CDSL had earnings beat, which other FINSERV stocks moved?
PRECEDED	NewsEvent → TradeDecision	lag_days	What news patterns precede winning trades in this sector?
MADE_BY	TradeDecision → Agent	confidence, was_correct	Which agent has the best track record in PHARMA?
CORRELATES_WITH	Stock → Stock	correlation_60d	When HDFC Bank moved, which other stocks moved in same direction?
FOLLOWED_BY	MacroRegime → Catalyst	frequency, avg_lag_days	In RISK_ON regimes, which catalysts appear most often?
CAUSED	Agent → Mistake	severity, recovered	Track agent error history for self-correction
LEARNED_FROM	Agent → Mistake	lesson_text, date	What has the agent explicitly learned from its mistakes?

Graph retrieval examples

```
// "What historically happens after delivery spikes in FINSERV stocks?"
MATCH (s:Stock)-[:BELONGS_TO]->(:Sector {name:"FINSERV"})
      <-[:TRIGGERED]-(c:Catalyst {type:"DELIVERY_SPIKE"})
      -[:PRECEDED]->(td:TradeDecision)
WHERE td.pnl_outcome IS NOT NULL
RETURN avg(td.pnl_outcome) as avg_return,
```

```
       count(td) as sample_size,
       stdev(td.pnl_outcome) as volatility

// "Has Agent QUANTRA made similar mistakes before in PHARMA?"
MATCH (a:Agent {name:"QUANTRA"})-[:CAUSED]->(m:Mistake)
      -[:RELATED_TO]->(s:Stock)-[:BELONGS_TO]->(:Sector {name:"PHARMA"})
RETURN m.error_type, m.description, m.date ORDER BY m.date DESC LIMIT 5
```

Neo4j Community Edition limit

Community Edition is free but single-instance only. For HIVEMIND's scale (hundreds of nodes, thousands of edges), this is entirely sufficient. The graph grows slowly — roughly 10-30 new nodes per nightly run. At this rate you have years of headroom before needing a paid tier.

3. Hybrid Retrieval Engine

Pure vector search is insufficient for financial reasoning. A query like "delivery spike in FINSERV after FII outflow" needs exact keyword matching (BM25) to find the literal term "delivery spike", semantic search to find related concepts ("institutional accumulation", "block deal"), and graph traversal to find FINSERV stocks specifically. The retrieval engine runs all three in parallel and fuses the results.

> **Core retrieval pattern**
> Query → [BM25 + Dense Vector + Graph] in parallel → RRF Fusion → Cross-encoder Reranking → Context Compression → LLM context injection. This pattern is used for every retrieval call in every agent.

3.1 Retrieval Pipeline — Step by Step

Step 1: Query rewriting

Raw agent queries are almost always too narrow. Before retrieval, the Memory Manager rewrites every query into 3-4 sub-queries using a fast small model (Groq Llama-3.1-8B, ~200ms). This step alone improves retrieval recall by 30-50% in financial text.

```
# Example query rewriting
original: "delivery spike CDSL"

rewritten to:
  q1: "CDSL delivery ratio spike institutional accumulation"
  q2: "depositories NSDL CDSL block deal volume surge"
  q3: "CDSL FINSERV momentum breakout"
  q4: "delivery spike outcome winning trade FINSERV historical"
```

Step 2: Parallel retrieval — three channels

```
async def retrieve_parallel(queries: list[str], ticker: str, sector: str):
    bm25_task   = asyncio.create_task(bm25_search(queries, ticker, sector))
    dense_task  = asyncio.create_task(dense_search(queries, ticker, sector))
    graph_task  = asyncio.create_task(graph_search(ticker, sector))
    bm25_res, dense_res, graph_res = await asyncio.gather(
        bm25_task, dense_task, graph_task
    )
    return bm25_res, dense_res, graph_res
```

Channel	Implementation	Index	Returns	Strengths
BM25	rank_bm25 Python library OR PostgreSQL ts_rank	TimescaleDB full-text index on thesis + news text fields	Top 30 matching text chunks by keyword overlap	Exact term matching: ticker symbols, catalyst names, financial terms
Dense vector	Qdrant cosine similarity	BAAI/bge-m3 embeddings (1024d) in all 6 collections	Top 30 semantically similar embeddings	Conceptual similarity: "earnings beat" ↔ "profit exceeds estimate"

Channel	Implementation	Index	Returns	Strengths
			across collections	
Graph	Neo4j Cypher traversal	Relationship index on TRIGGERED, PRECEDED, CORRELATES_WITH	Structured fact chains: entity relationships and historical patterns	Multi-hop reasoning: stock → catalyst → historical_outcome chains

Step 3: RRF Fusion

Reciprocal Rank Fusion merges the three ranked lists into a single unified ranking without requiring score normalisation. It is the correct fusion algorithm for heterogeneous retrieval channels.

```
def rrf_fusion(bm25_results, dense_results, graph_results, k=60):
    """Reciprocal Rank Fusion. k=60 is standard."""
    scores = defaultdict(float)
    for results in [bm25_results, dense_results, graph_results]:
        for rank, doc in enumerate(results):
            scores[doc.id] += 1.0 / (k + rank + 1)
    return sorted(scores.items(), key=lambda x: x[1], reverse=True)[:100]
```

Step 4: Cross-encoder reranking

The fused top-100 candidates are reranked with a cross-encoder model. Unlike bi-encoders (which embed query and doc separately), cross-encoders see query + doc together and produce a much more precise relevance score. This is the highest quality step and is worth the extra 200-400ms.

```
from sentence_transformers import CrossEncoder

reranker = CrossEncoder("BAAI/bge-reranker-v2-m3")  # free, local, multilingual

def rerank(query: str, candidates: list[Document]) -> list[Document]:
    pairs = [(query, doc.text) for doc in candidates]
    scores = reranker.predict(pairs, batch_size=32)
    ranked = sorted(zip(scores, candidates), reverse=True)
    return [doc for _, doc in ranked[:10]]  # top 10 for context
```

Step 5: Context compression

The top 10 reranked documents are still too long to inject raw into an LLM context. Context compression extracts only the relevant sentences from each document, reducing token usage by 60-80% while preserving the key facts. This is the BEE-RAG principle: entropy dilution from oversized contexts actively hurts reasoning quality.

```
async def compress_context(query: str, docs: list[Document]) -> str:
    """Extract only the sentences that directly answer the query."""
    compression_prompt = f"""
    Query: {query}
    Documents: {format_docs(docs)}
    Extract only the sentences that directly answer the query.
    Output a JSON list of extracted sentences. Nothing else."""
    # Use Groq Llama-3.1-8B: fast + cheap for compression
```

```
    result = await groq_complete(compression_prompt, model="llama-3.1-8b-instant")
    return " ".join(json.loads(result))
```

3.2 Iterative Retrieval (Re-retrieve on gap detection)

A single retrieval pass is never enough for a trade decision. After the first synthesis attempt, the agent identifies what it still does not know and runs a second targeted retrieval. This iterative pattern is what separates serious agentic RAG from basic retrieval.

```
async def iterative_retrieve(ticker, sector, working_memory):
    # Round 1: broad retrieval
    context_r1 = await retrieve_and_compress(
        queries=rewrite_queries(f"{ticker} {sector} factor analysis"),
        ticker=ticker, sector=sector
    )
    # Identify gaps: what is the agent still uncertain about?
    gaps = await identify_knowledge_gaps(context_r1, working_memory)
    # gaps example: ["USFDA status for CDSL?", "promoter selling recent?"]

    if not gaps:
        return context_r1

    # Round 2: targeted gap-filling retrieval
    context_r2 = await retrieve_and_compress(
        queries=gaps, ticker=ticker, sector=sector
    )
    return merge_contexts(context_r1, context_r2)
```

4. Agent Personalities & Specifications

HIVEMIND operates a swarm of seven agents. Each agent has a name, a defined personality archetype, explicit cognitive biases (which are features, not bugs), a specific memory access pattern, and a defined output schema. The orchestrator runs a structured adversarial debate between agents before producing any final output.

Agent personalities are permanent prompt fixtures

The personality, biases, and voice of each agent are part of the system prompt and never change run-to-run. What changes is the episodic memory injected into context (recent mistakes, recent decisions). The personality is stable; the knowledge is dynamic.

4.1 The Seven Agents

VEGA — Macro Regime Analyst

`Model: Gemini 1.5 Flash (Google AI Studio) — 1,500 req/day`

Personality: Detached, probabilistic, deeply sceptical of narratives. Speaks in percentiles and regime probabilities. Never says "the market will go up" — says "RISK_ON regime with 72nd VIX percentile implies elevated tail risk." Has seen too many bull markets reverse to be an optimist.

Biases / priors: Strongly weighs FII flows over domestic flows. Believes VIX is the single most honest signal in Indian markets. Distrusts RBI forward guidance. Has a prior that market tops happen when everyone is confident.

Memory access: T2 (last 30 days of regime signals), T3 (market_knowledge collection), T5 (MacroRegime nodes)

Outputs: regime_signal (RISK_ON/CAUTIOUS/RISK_OFF), confidence_pct, key_risk_factors[], macro_thesis (200 words), halt_recommendation (bool)

NYSA — News & Catalyst Analyst

`Model: Groq Llama-3.1-70B — 14,400 req/day, ultra-fast for real-time NLP`

Personality: Hypervigilant, pattern-obsessed, slightly paranoid. Reads between lines of corporate announcements. Assumes that any news release is incomplete and looks for what is NOT said. Has a nose for governance issues — picks up on subtle language shifts in management commentary.

Biases / priors: Overweights negative news relative to positive (loss aversion prior). Distrusts "one-off item" explanations. Treats management guidance cuts as more informative than beats. Has a sector-specific vocabulary: knows that "sluggish demand environment" in FMCG means inventory pile-up.

Memory access: T2 (last 7 days of news episodes for ticker + sector), T3 (news_episodes collection), T5 (NewsEvent and Catalyst nodes)

Outputs: sentiment_score (-1 to +1), catalyst_tags[], red_flag (bool), red_flag_severity (0-1), news_summary (100 words), retrieved_similar_events[]

QUANTRA — Quantitative Factor Analyst

`Model: NVIDIA Nemotron-70B (NIM) — deterministic, temperature=0.0`

Personality: Rigidly data-driven, almost autistic about precision. Refuses to use qualitative language unless backed by a specific number. Deeply sceptical of narrative-driven investing. Believes factor exposure explains 90%+ of returns and the rest is noise. Talks in standard deviations, not adjectives.

Biases / priors: Trusts idiosyncratic momentum (OLS epsilon) above all other signals. Believes delivery ratio spikes are the single most honest signal in NSE markets because they are hard to fake. Deeply distrusts P/E ratios as valuation tools — prefers EV/FCF. Automatically discounts any stock where OI is rising with falling price.

Memory access: T2 (last 5 factor score snapshots for ticker), T3 (quant_signals collection), T4 (agent_procedures — learned factor weights per sector)

Outputs: factor_attribution (JSON: each param with sub-score and reasoning), composite_score (0-1), timeframe_recommendation (monthly/quarterly/annual), invalidation_conditions[], entry_logic_paragraph (150 words)

LEXA — Fundamental Researcher

`Model: Gemini 1.5 Pro + Google Search grounding — used sparingly (max 10 calls/night)`

Personality: Patient, long-horizon, Buffett-influenced. Genuinely interested in business quality over price action. Will refuse to recommend a stock that has great momentum but terrible fundamentals — explicitly states this. Writes longer, more nuanced outputs than other agents. Has strong opinions on management quality.

Biases / priors: Strongly weights ROCE consistency over 5+ years (not just current). Treats promoter holding as a proxy for conviction, not control. Has a "moat checklist" — pricing power, switching costs, network effects, cost advantage — and mentally runs every stock through it. Distrusts companies that grow revenue faster than cash flow.

Memory access: T2 (last 3 research notes for ticker), T3 (research_notes collection), T5 (Stock and Catalyst nodes for fundamental context)

Outputs: moat_score (0-5), quality_assessment (paragraph), risk_flags[], valuation_band (JSON: bear/base/bull target), fundamental_thesis (300 words), red_flag (bool)

SECTORA — Sector Rotation Specialist

`Model: Groq Llama-3.1-70B (sector-specific system prompts, cached)`

Personality: Thinks in cycles, not stocks. Sees individual tickers as just expressions of sector-level forces. Compares every stock against its sector peers before forming an opinion. Has deep sector-specific knowledge baked into different system prompt variants (6 sector prompts, one per sector queue).

Biases / priors: Believes sector rotation is the dominant force in NSE mid-caps. Overweights relative strength vs sector benchmark over absolute momentum. Thinks intra-sector leadership changes are more important than macro calls. Has strong priors on which sectors lead and which lag in each regime phase.

Memory access: T2 (last 14 days of sector episodes), T3 (trade_theses — filtered by sector), T5 (Sector and CORRELATES_WITH relationships)

Outputs: sector_outlook (BULLISH/NEUTRAL/BEARISH), leadership_stocks[], laggard_stocks[], rotation_thesis (150 words), sector_risk_factors[]

VERA — Critic & Verification Agent

`Model: Gemini 1.5 Flash — runs AFTER all other agents, never in parallel`

Personality: Adversarial by design. VERA's only job is to find holes in the other agents' arguments. Has no positive thesis of its own — purely destructive. Asks: "What would have to be true for this call to be wrong?" and then checks whether any evidence supports that. VERA can and does veto a PROCEED decision.

Biases / priors: Assumes every PROCEED recommendation has at least one overlooked risk. Has a checklist of common agent failure modes: recency bias, narrative anchoring, data-mining, correlation-as-causation. Particularly suspicious of high-confidence calls — in markets, high confidence often precedes large losses.

Memory access: T2 (all agents' recent mistake logs), T3 (post_mortems collection — specifically failed trades), T5 (Mistake nodes for all agents)

Outputs: veto (bool), veto_reason (text if veto=true), risk_score (0-1), unchecked_risks[], confidence_adjustment (-0.3 to 0), verification_report (200 words)

APEX — Orchestrator & Decision Maker

`Model: Gemini 1.5 Pro — one call per surviving ticker, after all other agents complete`

Personality: Senior portfolio manager archetype. Has seen bull markets, bear markets, and everything in between. Respects data but is not enslaved to it. Willing to override a strong quant signal if the fundamentals are broken, or override a red flag if it is already priced in. Makes final calls clearly and owns them.

Biases / priors: Weights VERA's critique heavily — a veto from VERA requires strong counter-evidence from at least two other agents to override. Believes a 2:1 R:R is the minimum for any trade, non-negotiable. Has a strong prior against concentration — will never approve more than 2 picks per sector per night. Dislikes uncertainty — if agents strongly disagree, defaults to SKIP.

Memory access: T2 (all agents' current outputs), T3 (trade_theses — all historical decisions for cross-reference), T4 (orchestrator override thresholds), T5 (full graph neighbourhood)

Outputs: decision (PROCEED/SKIP), entry_price, stop_price, target_price, timeframe, confidence (0-1), final_thesis (250 words), agent_agreement_score, dissenting_agent (if any), vector_id (written to T3)

4.2 Agent Debate Protocol

When agents disagree significantly (defined as: any agent's output conflicts with the majority direction by > 0.3 on a 0-1 confidence scale), APEX runs a structured debate round before making the final call. This is not a prompt trick — it is a literal second LLM call where APEX is shown all conflicting outputs and asked to adjudicate.

```
# Debate trigger logic
def should_debate(agent_outputs: dict) -> bool:
    signals = [out["bullish_score"] for out in agent_outputs.values()]
    return max(signals) - min(signals) > 0.3  # significant disagreement

# Debate prompt structure fed to APEX
DEBATE_TEMPLATE = """
You are APEX, a senior portfolio manager.
```

```
The following analysts disagree on {ticker}. You must adjudicate.

BULLISH CASE (agents: {bullish_agents}):
{bullish_arguments}

BEARISH CASE (agents: {bearish_agents}):
{bearish_arguments}

VERA (critic) says: {vera_output}

Historical base rate for similar disagreements:
{graph_retrieved_similar_debates}

Adjudicate. State which case is stronger and why.
Then output your final decision as JSON.
"""
```

5. Mistake Learning System

This is the component that makes HIVEMIND genuinely intelligent over time rather than just fast. Every closed trade triggers a structured post-mortem. Every post-mortem produces a mistake record if the trade failed, or a success record if it worked. These records are injected into future agent runs, creating a closed learning loop.

5.1 Post-Mortem Trigger

The Feedback Agent (Python) runs automatically when a paper trade is closed by the execution engine. It receives: the closed trade record, the agent_output that generated the recommendation, the factor_scores at time of entry, and the full working memory log from the original run (stored in Redis T2 for 7 days).

```
class FeedbackAgent:
    """Runs on every closed trade. Attributes outcome to specific signals."""

    async def run(self, trade_id: str):
        trade     = await db.fetch_trade(trade_id)
        ao        = await db.fetch_agent_output(trade.agent_output_id)
        scores    = await db.fetch_factor_scores_at(trade.symbol, trade.entry_time)
        wm_log    = await redis.get(f"wm_log:{ao.run_id}")  # working memory log

        # Semantic search: find similar historical setups
        similar = await qdrant.search("post_mortems",
            query=f"{trade.symbol} {ao.sector} {ao.thesis_summary}",
            top_k=5)

        # Graph query: find similar debate outcomes
        graph_ctx = await neo4j.query(SIMILAR_DEBATE_QUERY,
            sector=ao.sector, outcome=trade.exit_reason)

        post_mortem = await self._analyse(trade, ao, scores, similar, graph_ctx)
        await self._write_to_all_memory_tiers(post_mortem)
```

5.2 Post-Mortem Output Schema

```
PostMortem = {
    "trade_id":       str,
    "outcome":        "WIN" | "LOSS" | "BREAKEVEN",
    "pnl_pct":        float,
    "exit_reason":    "SL_HIT" | "TARGET" | "TIMEOUT",

    # Attribution: which signals were right/wrong
    "signal_attribution": {
        "delivery_spike":     {"was_present": bool, "was_predictive": bool},
        "vcp_flag":           {"was_present": bool, "was_predictive": bool},
        "news_sentiment":     {"score": float,      "was_predictive": bool},
        "fundamental_moat":   {"score": float,      "was_predictive": bool},
        "sector_rotation":    {"was_present": bool, "was_predictive": bool},
    },

    # Agent attribution: who was right, who was wrong
    "agent_attribution": {
        "VEGA":    {"was_correct": bool, "confidence_at_call": float},
```

```
        "NYSA":    {"was_correct": bool, "confidence_at_call": float},
        "QUANTRA": {"was_correct": bool, "confidence_at_call": float},
        "LEXA":    {"was_correct": bool, "confidence_at_call": float},
        "VERA":    {"flagged_correctly": bool, "veto_overridden": bool},
    },

    # Learning outputs
    "what_worked":    str,   # specific signals that predicted outcome
    "what_failed":    str,   # specific signals that were wrong
    "lesson":         str,   # one-line actionable takeaway
    "mistake_type":   str | None,  # if LOSS: classify error type
}
```

5.3 Mistake Classification Taxonomy

Mistake type	Definition	Agent most likely responsible	Corrective action
MISSED_RED_FLAG	Trade failed due to a risk that was available in the data but not flagged	NYSA or LEXA	Increase red_flag sensitivity weight in T4 procedural memory for this sector
REGIME_MISMATCH	Entered a long in deteriorating macro conditions	VEGA	Raise VEGA's override authority in APEX debate prompt
FACTOR_FALSE_POSITIVE	Quant score high but fundamental quality poor — factor signal didn't translate	QUANTRA + LEXA	Increase LEXA moat_score weight in composite formula for this sector
NARRATIVE_ANCHORING	Agents committed to a bull thesis and dismissed contradicting evidence	APEX debate	Require VERA to cite 2+ contradicting signals before APEX can override
PREMATURE_EXIT	Stop loss hit on noise, stock continued higher	Execution engine	Widen stop to 2.5% in low-volatility regimes (VCP confirmed)
CONCENTRATION	Two losing trades in same sector on same night	APEX	Enforce hard limit: max 1 new position per sector per night
STALE_FUNDAMENTAL	Research note used was > 90 days old	LEXA	Force fresh Screener.in pull if last research note > 60 days

5.4 How mistakes propagate into future runs

After classification, the Feedback Agent writes the mistake to three places simultaneously:

1. Redis T2: agent mistake log — injected into the responsible agent's system prompt on the next run as "Recent errors to avoid."

2. Qdrant T3: post_mortems collection — embedded and stored permanently, retrieved semantically by all agents when similar setups appear.
3. Neo4j T5: Mistake node linked to Agent, Stock, Sector, and Catalyst nodes — enables graph queries like "has QUANTRA over-called delivery spikes in PHARMA before?"
4. PostgreSQL T4: if the same mistake type appears 3+ times, the relevant factor weight in agent_procedures is automatically adjusted by ±0.05, capped at bounds.

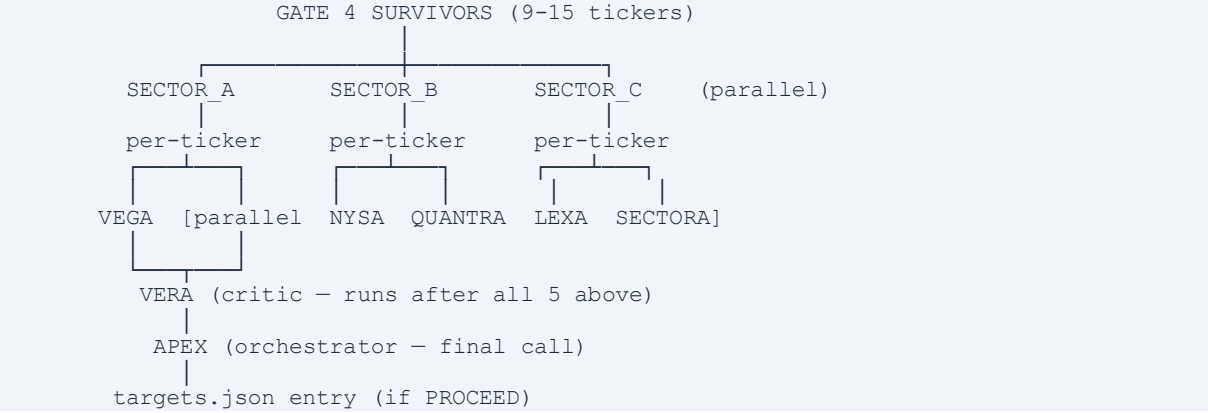
The result

An agent that repeatedly misses USFDA alerts in PHARMA stocks will, after 3 misses, have its USFDA-related keywords automatically upweighted in its BM25 retrieval query, its news_episodes retrieval expanded to include USFDA-specific sub-queries, and a standing reminder injected into every PHARMA run: "You have missed USFDA alerts 3 times. Explicitly check CDSCO and FDA import alert databases before any PHARMA recommendation."

6. Swarm Execution Model

The swarm runs as a directed acyclic graph of async tasks. Sector queues are fully parallel. Within each sector queue, the first five agents (VEGA, NYSA, QUANTRA, LEXA, SECTORA) run in parallel per stock. VERA runs after all five complete. APEX runs last, after VERA. This ordering is non-negotiable — critics must run after generators.

6.1 Execution DAG



6.2 Rate Limit Management

With 9-15 tickers, 7 agents per ticker (5 parallel + 2 sequential), the worst case is 105 LLM calls per nightly run. This must be distributed carefully across providers to stay within free tier limits.

Provider	Daily free limit	Calls allocated per night	Headroom	Overflow fallback
Google AI Studio (Flash)	1,500 req/day	VEGA: 1 macro call APEX: 15 max VERA: 15 max Total: ~31	1,469 remaining	N/A — well within limits
Google AI Studio (Pro)	50 req/day	LEXA: max 15 deep research calls	35 remaining	Downgrade to Flash if >35 tickers survive Gate 4
Groq (Llama-3.1-70B)	14,400 req/day	NYSA: 15 news calls SECTORA: 15 calls Query rewriting: 60 (4 per ticker) Total: ~90	14,310 remaining	Cerebras (30 req/min free)
Groq (Llama-3.1-8B)	14,400 req/day	Context compression: ~90 calls Gap identification: ~30 calls Total: ~120	14,280 remaining	OpenRouter free models

Provider	Daily free limit	Calls allocated per night	Headroom	Overflow fallback
NVIDIA NIM	1,000 credits/month	QUANTRA: 15 calls/night = ~450/month	550 credits/month	OpenRouter: Nemotron variant

6.3 Concurrency Control

```
import asyncio
from asyncio import Semaphore

# Semaphores to prevent rate limit bursts
GROQ_SEM   = Semaphore(5)    # max 5 concurrent Groq calls
GEMINI_SEM = Semaphore(3)    # max 3 concurrent Gemini calls
NIM_SEM    = Semaphore(2)    # max 2 concurrent NIM calls

async def safe_groq_call(prompt, **kwargs):
    async with GROQ_SEM:
        await asyncio.sleep(0.1)  # 100ms spacing
        return await groq_client.chat.completions.create(
            model="llama-3.1-70b-versatile",
            **kwargs,
            messages=[{"role":"user","content":prompt}]
        )
```

6.4 Agent Output Validation

Every agent output is validated against a Pydantic schema before being accepted. Invalid outputs trigger a retry with an error correction prompt. Maximum 2 retries per agent per ticker. On third failure, the agent's output is marked as UNAVAILABLE and APEX is notified to proceed with reduced information (and lower confidence ceiling of 0.6 in that case).

```
from pydantic import BaseModel, Field, validator

class NewsAnalystOutput(BaseModel):
    sentiment_score:    float = Field(ge=-1.0, le=1.0)
    catalyst_tags:      list[str] = Field(max_items=10)
    red_flag:           bool
    red_flag_severity:  float = Field(ge=0.0, le=1.0)
    news_summary:       str = Field(max_length=500)

    @validator("catalyst_tags")
    def tags_must_be_known(cls, v):
        VALID_TAGS = {"EARNINGS_BEAT","EARNINGS_MISS","MGMT_CHANGE",
                      "USFDA_ALERT","ORDER_WIN","BLOCK_DEAL","PLEDGE"}
        return [t for t in v if t in VALID_TAGS]
```

7. Long-Horizon Memory & Context Management

The hardest problem in agentic AI systems is not the intelligence of individual agents — it is maintaining coherent, relevant, long-term context without filling the context window with noise. HIVEMIND solves this with three mechanisms: progressive summarisation, context budgeting, and tiered memory injection.

7.1 Progressive Summarisation

Raw agent outputs from 90+ days ago are not stored at full length in active retrieval. The system runs a weekly summarisation job that compresses older T3 records while preserving their semantic meaning. This is analogous to how humans remember: the specific words of a conversation fade, the meaning and lesson persist.

Age of record	Storage format	Retrieved length	Summarisation model
0–7 days	Full agent output (500-1,000 words)	Full text	No compression — too recent
7–30 days	Compressed summary (150-200 words)	Summary only	Groq Llama-3.1-8B weekly batch job
30–90 days	Key facts only (50-80 words)	Key facts	Groq Llama-3.1-8B monthly batch job
90+ days	Structured JSON (decision, outcome, lesson)	~30 words	Extracted fields only, no prose

7.2 Context Budget per Agent Call

Every agent has a defined context budget in tokens. The Memory Manager is responsible for filling that budget optimally across the five memory tiers. It never exceeds the budget. If the budget is tight, it prioritises: T2 recent episodes > T3 similar setups > T5 graph facts > T4 procedural rules.

Agent	Total context budget	T1 working	T2 recent	T3 semantic	T5 graph	T4 procedural	System prompt
VEGA	8,000 tok	1,000	2,000	2,000	1,000	500	1,500
NYSA	6,000 tok	500	2,500	1,500	500	300	700
QUANTRA	6,000 tok	1,500	1,000	1,500	800	700	500
LEXA	10,000 tok	1,000	1,500	3,000	1,500	500	2,500

Agent	Total context budget	T1 working	T2 recent	T3 semantic	T5 graph	T4 procedural	System prompt
SECTORA	6,000 tok	800	2,000	1,500	800	400	500
VERA	12,000 tok	2,000	2,000	3,000	1,500	500	3,000
APEX	16,000 tok	3,000	2,000	3,000	2,000	1,000	5,000

7.3 The Memory Manager

The Memory Manager is a Python class that runs before every agent call. It queries all relevant memory tiers, compresses and ranks the results, and assembles the final context package within the token budget. Agents never call memory directly.

```
class MemoryManager:
    def __init__(self, ticker, sector, agent_name, budget_tokens):
        self.ticker = ticker
        self.sector = sector
        self.agent  = agent_name
        self.budget = budget_tokens

    async def assemble_context(self, query: str) -> ContextPackage:
        # Parallel memory retrieval across tiers
        t2, t3, t5, t4 = await asyncio.gather(
            self._fetch_t2_episodes(),
            self._fetch_t3_semantic(query),
            self._fetch_t5_graph(),
            self._fetch_t4_procedures(),
        )
        # Compress and rank within budget
        return self._assemble_within_budget(t2, t3, t5, t4)

    def _assemble_within_budget(self, t2, t3, t5, t4) -> ContextPackage:
        """Priority: T2 > T3 > T5 > T4. Fill budget greedily."""
        package = ContextPackage()
        remaining = self.budget - SYSTEM_PROMPT_TOKENS[self.agent]

        for tier, weight in [(t2,0.35),(t3,0.35),(t5,0.20),(t4,0.10)]:
            alloc = int(remaining * weight)
            package.add(tier, max_tokens=alloc)

        return package
```

8. Developer Sprint Plan

This section is the primary reference for the assigned developer. It is structured as 10 two-week sprints covering 20 weeks, from environment setup through to a fully operational AI layer with long-term memory, graph intelligence, and adaptive learning. Each sprint has specific tasks, acceptance criteria, and dependencies.

Prerequisites before Sprint 1

The base data pipeline (HIVEMIND Technical Specification v1.0, Layers 1-4) must be complete and verified. The developer needs: TimescaleDB running with data flowing, Gate 4 factor scores generating nightly, and targets.json producing 9-15 daily candidates before any sprint in this document begins.

8.1 Sprint Overview

Sprint	Name	Weeks	Core deliverable	Key risk
S1	Infrastructure setup	1–2	Neo4j + Qdrant collections + Redis schema + Memory Manager skeleton	Neo4j Docker memory config on low-RAM machines
S2	Hybrid retrieval engine	3–4	BM25 + dense + graph retrieval working; RRF fusion; cross-encoder reranking verified	Latency: full pipeline must complete < 2s per ticker
S3	Query rewriting + iterative retrieval	5–6	Query rewriting with Llama-3.1-8B; gap detection; 2-round iterative retrieve verified	Rate limit budgeting with rewrite calls added
S4	VEGA + NYSA agents live	7–8	Both agents running against real nightly data, outputs validated, T2/T3 writes confirmed	NYSA news RSS parsing edge cases (malformed XML)
S5	QUANTRA + LEXA + SECTORA agents live	9–10	All five generator agents running in parallel, sector routing confirmed	Gemini Pro rate limit — LEXA must batch efficiently
S6	VERA critic agent + APEX orchestrator	11–12	Full 7-agent swarm producing targets.json nightly, debate protocol working	APEX prompt engineering — JSON output stability
S7	Knowledge graph population	13–14	Neo4j nodes + relationships populated from last 90 days of trade history; Cypher queries verified	Graph schema migration if T3 data needs restructuring
S8	Mistake learning + feedback loop	15–16	Post-mortem agent running on every close; T2/T3/T4/T5 all updated; agent prompts receiving mistake context	Feedback loop creating circular errors (overcorrection)

Sprint	Name	Weeks	Core deliverable	Key risk
S9	Progressive summarisation + context budgeting	17–18	Memory Manager assembling optimal context packages; old records compressed; budget enforced per agent	Token counting precision — off-by-one errors in budget
S10	Evaluation, tuning, and live paper trading	19–20	30-day live paper run with full AI layer; agent attribution tracked; weight optimisation running	First real data may expose retrieval quality gaps

8.2 Sprint 1 — Infrastructure Setup (Weeks 1–2)

Task ID	Task + detail	Effort	Depends on
S1-T1	Set up Neo4j Community Edition via Docker docker run -d --name neo4j -p 7474:7474 -p 7687:7687 -e NEO4J_AUTH=neo4j/hivemind neo4j:5-community. Verify browser at localhost:7474. Run test Cypher query.	2h	None
S1-T2	Create Neo4j schema and indexes Create all node labels and relationship types per Section 2.5. Create indexes on :Stock(symbol), :TradeDecision(run_id). Write migration script.	4h	S1-T1
S1-T3	Create all 6 Qdrant collections trade_theses, news_episodes, research_notes, quant_signals, post_mortems, market_knowledge. Vector size: 1024 for bge-m3 collections, 384 for MiniLM. Distance: cosine.	3h	Qdrant Cloud account
S1-T4	Install and test BAAI/bge-m3 locally pip install sentence-transformers. Download BAAI/bge-m3. Write test: embed 10 financial sentences, verify cosine similarity ordering is sensible.	2h	GPU/CPU with 8GB RAM
S1-T5	Install and test BAAI/bge-reranker-v2-m3 pip install sentence-transformers. Load cross-encoder. Write test: given 20 candidate texts, verify top-3 reranked are most relevant to a sample query.	2h	S1-T4
S1-T6	Set up Redis schema for episodic memory Define all key patterns per Section 2.2. Write RedisEpisodicStore class with read/write/expire methods. Write unit tests for TTL behaviour.	4h	Redis Docker running
S1-T7	Build Memory Manager skeleton	4h	S1-T3,S1-T4,S1-T6

Task ID	Task + detail	Effort	Depends on
	Python class per Section 7.3. Stub out all tier methods (return empty lists). Wire constructor. Add logging to retrieval log in working_memory.		
S1-T8	Backfill historical data into T3 Embed and upsert last 90 days of existing trade decisions (from TimescaleDB agent_outputs) into Qdrant trade_theses collection. Write one-time migration script.	6h	S1-T3, existing data

S1 acceptance criteria

Neo4j browser shows all node labels. 6 Qdrant collections exist and accept test upserts. Redis episodic writes survive a restart test. Memory Manager can be instantiated without errors (stubs returning empty is fine).

8.3 Sprint 2 — Hybrid Retrieval Engine (Weeks 3–4)

Task ID	Task + detail	Effort	Depends on
S2-T1	Implement BM25 search over TimescaleDB text fields Add full-text search index: CREATE INDEX ON agent_outputs USING gin(to_tsvector('english', thesis || ' ' || risk_notes)). Write BM25SearchEngine class using ts_rank. Returns top 30 with doc_id and score.	6h	S1 complete
S2-T2	Implement dense vector search across Qdrant collections Write DenseSearchEngine class. For a query string: embed with bge-m3, run parallel search across all 6 collections, merge results with source tag. Returns top 30 with payload.	6h	S1-T3,S1-T4
S2-T3	Implement graph retrieval via Neo4j Cypher Write GraphSearchEngine class. For a (ticker, sector) pair: run the 3 pre-defined Cypher queries from Section 2.5. Returns structured fact list, not embeddings.	8h	S1-T2
S2-T4	Implement RRF fusion Write rrf_fusion(bm25_results, dense_results, graph_results, k=60). Returns unified ranked list with source breakdown per document.	3h	S2-T1,S2-T2,S2-T3
S2-T5	Implement cross-encoder reranking	4h	S1-T5, S2-T4

Task ID	Task + detail	Effort	Depends on
	Write CrossEncoderReranker class using bge-reranker-v2-m3. Takes fused top-100, query string, returns top 10 with rerank scores.		
S2-T6	Implement context compression Write ContextCompressor class using Groq Llama-3.1-8B per Section 3.1 Step 5. Must reduce 10 docs to < 800 tokens while preserving key facts.	6h	S2-T5
S2-T7	Wire retrieval pipeline end-to-end Wire: query → BM25+dense+graph (parallel) → RRF → rerank → compress → context package. Add timing logs. Full pipeline must complete < 2,000ms per ticker.	4h	S2-T6
S2-T8	Retrieval quality evaluation Write eval script: 20 test queries with known correct answers. Measure: recall@10 for BM25 alone, dense alone, hybrid combined. Hybrid must beat both alone.	8h	S2-T7

Critical performance gate

Sprint 2 does not proceed to Sprint 3 until the full retrieval pipeline completes under 2,000ms for a single ticker and hybrid recall@10 beats pure vector recall@10. Measure both before moving on.

8.4 Sprint 3 — Query Rewriting + Iterative Retrieval (Weeks 5–6)

Task ID	Task + detail	Effort	Depends on
S3-T1	Build query rewriting module QueryRewriter class: takes raw query string, calls Groq Llama-3.1-8B with rewrite prompt, returns 3-4 expanded sub-queries as JSON list. Include financial domain vocabulary in prompt.	5h	Groq API access
S3-T2	Build knowledge gap detector GapDetector class: takes first-round context + working_memory, calls Groq Llama-3.1-8B, returns list of specific unanswered questions. Output is 2-5 targeted query strings.	6h	S3-T1
S3-T3	Implement iterative retrieval loop Wire per Section 3.2: round1 retrieve → gap detect → conditional round2 retrieve → merge	4h	S3-T2, S2-T7

Task ID	Task + detail	Effort	Depends on
	contexts. Add skip logic: if gap_count == 0, skip round 2.		
S3-T4	Rate limit tracking for rewrite calls Add counter tracking Groq-8B calls per nightly run. If count > 200, switch to lighter query expansion via synonym lookup (no LLM). Log all rate limit events.	3h	S3-T3
S3-T5	Retrieval pipeline integration test Run full pipeline (rewrite → retrieve × 2 → rerank → compress) against 5 live tickers from Gate 4 output. Verify context quality manually: does the context package contain relevant, non-redundant facts?	6h	S3-T4
S3-T6	Wire Memory Manager with full retrieval pipeline Replace stub methods in Memory Manager with real implementations from S2 + S3. Test: Memory Manager for QUANTRA agent on CDSL returns a context package with < 1,500 tokens and contains factor history.	6h	S3-T5

8.5 Sprints 4–6 — Agent Implementation (Weeks 7–12)

Sprints 4, 5, and 6 each follow the same pattern for each agent: write system prompt, implement agent class, wire to Memory Manager, validate output schema, run against 5 live tickers, review output quality manually. The detailed task breakdown follows the pattern below — replicate for each agent.

Task template	Detail	Effort per agent
Write system prompt	Implement the agent's personality, biases, and voice exactly per Section 4.1 spec. Include: persona paragraph, explicit biases list, output format specification, failure modes to avoid. Test prompt stability: 10 identical inputs should produce structurally identical JSON outputs.	8h
Implement agent class	Python class with async run() method. Wire to Memory Manager. Call appropriate LLM provider. Parse and validate output with Pydantic schema. Handle retries (max 2) on validation failure.	6h
Write T2/T3/T5 on completion	After successful run(), write episode to Redis (T2), embed and upsert to Qdrant (T3), and write nodes/edges to Neo4j (T5). All three writes must be atomic — use try/except with rollback logging.	4h
Live data validation	Run agent against 5 tickers from current Gate 4 output. Read outputs manually. Check: Is the personality	4h

Task template	Detail	Effort per agent
	consistent? Is the output JSON valid? Is the reasoning grounded in retrieved context (not hallucinated)? Are sector-specific nuances reflected?	
Mistake injection test	Manually inject a fake mistake into Redis (agent mistake log). Re-run agent on same ticker. Verify the mistake is visible in the system prompt and the agent acknowledges it in its reasoning.	3h

8.6 Sprint 7 — Knowledge Graph Population (Weeks 13–14)

Task ID	Task + detail	Effort	Depends on
S7-T1	Graph writer service Write Neo4jWriter class: methods for creating all 8 node types and all 8 relationship types from Section 2.5. Use MERGE (not CREATE) to avoid duplicates. Idempotent writes.	8h	S1-T2
S7-T2	Backfill 90 days of trade history into graph Migration script: for every historical trade_log row, create Stock, TradeDecision, Agent, and MacroRegime nodes. Wire MADE_BY, TRIGGERED, FOLLOWED_BY relationships.	10h	S7-T1
S7-T3	Real-time graph update on every agent run Wire Neo4jWriter into all 7 agent classes. On every completed run: upsert Stock node, create TradeDecision node, wire agent edges. Test: run swarm, check graph has new nodes.	6h	S6 complete, S7-T1
S7-T4	Implement graph-based retrieval queries Implement the 3 Cypher queries from Section 2.5 as Python methods. Add to GraphSearchEngine (S2-T3). Verify: queries return non-empty results for CDSL + FINSERV.	5h	S7-T2
S7-T5	Graph visualisation sanity check Open Neo4j browser. Run MATCH (n) RETURN n LIMIT 100. Visually verify: nodes exist, relationships are correct, properties are populated. Screenshot for documentation.	2h	S7-T4

8.7 Sprint 8 — Mistake Learning + Feedback Loop (Weeks 15–16)

Task ID	Task + detail	Effort	Depends on
S8-T1	Implement Feedback Agent Python class per Section 5.1. Inputs: closed trade record, agent_output, factor_scores, Redis working memory log. Outputs: PostMortem schema (Section 5.2). Use Groq Llama-3.1-70B.	8h	S7 complete
S8-T2	Mistake classification logic Implement classify_mistake(post_mortem) function. Maps post_mortem fields to one of 7 mistake types (Section 5.3). Rule-based classifier, not LLM.	4h	S8-T1
S8-T3	Write mistakes to all 4 memory tiers Implement _write_to_all_memory_tiers() per Section 5.4. Redis: append to agent mistake log. Qdrant: upsert to post_mortems. Neo4j: create Mistake node + CAUSED/LEARNED_FROM edges. PostgreSQL: update agent_procedures if 3+ same mistake type.	8h	S8-T2
S8-T4	Wire feedback to execution engine In Node.js execution engine: on every position close event, publish to Redis channel "trade:closed:{trade_id}". Python Feedback Agent subscribes and triggers automatically.	4h	S8-T3
S8-T5	End-to-end learning test Manually create 3 fake closed trades with SL_HIT. Each tagged as MISSED_RED_FLAG for NYSA in PHARMA. Verify: after 3 runs, NYSA's system prompt for PHARMA now includes the mistake, Qdrant post_mortems has 3 records, T4 agent_procedures has updated usfda_alert_severity.	6h	S8-T4
S8-T6	Anti-overcorrection guard Implement correction dampening: weight adjustments from T4 are applied with a 0.5 learning rate (new = old + 0.5 * delta). Add min/max bounds check (no weight < 0.05 or > 0.50). Unit test with boundary inputs.	3h	S8-T5

8.8 Sprints 9–10 — Context Budgeting, Evaluation & Live Run (Weeks 17–20)

Task	Detail	Sprint	Effort
Implement token counter	Write TokenCounter class using tiktoken. Counts tokens for each context tier. Used by Memory	S9	3h

Task	Detail	Sprint	Effort
	Manager to enforce per-agent budgets (Section 7.2).		
Implement progressive summarisation job	Weekly cron job (Sunday 02:00 IST): query Qdrant for records 7-30 days old, summarise with Groq Llama-3.1-8B, update Qdrant payload. Monthly: further compress 30-90 day records.	S9	8h
Memory Manager budget enforcement	Replace greedy fill in Memory Manager with token-budget-aware assembly (Section 7.3). Test: with a 6,000-token budget for NYSA, assembled context must not exceed 6,000 tokens.	S9	5h
Build agent attribution dashboard	Streamlit page: for each closed trade, show which agent's signals were correct. Track 30-day rolling win rate per agent. Display on main dashboard.	S9	6h
30-day live paper simulation	Enable full AI layer for live nightly runs. Do not touch the code. Observe outputs daily. Log qualitative observations: are agent personalities consistent? Are mistakes being learned?	S10	Passive
Retrieval quality audit	After 30 days: randomly sample 20 completed agent runs. For each, check: was the retrieved context relevant? Were similar past trades retrieved correctly? Report recall and precision estimates.	S10	8h
Weight optimisation pass	Run scipy optimise over 30 days of closed trades. Find composite_score weights that maximise Sharpe. Compare to baseline weights. Update agent_procedures if improvement > 5%.	S10	6h
Regression test suite	Write pytest suite covering: Memory Manager budget, retrieval pipeline latency, all agent output schemas, debate trigger logic, feedback loop writes. Must pass before any future code change.	S10	10h

9. Evaluation Framework

A system this complex requires structured evaluation at three levels: retrieval quality (is the right context being found?), agent quality (are the agents reasoning correctly?), and system quality (is the swarm generating profitable trade ideas?). Each level has specific metrics and measurement methods.

9.1 Retrieval Quality Metrics

Metric	Definition	Target	Measurement method
Recall@10	Of the 10 most relevant documents, how many are in the top-10 retrieved?	> 0.75	Hand-label 30 test queries with gold-standard relevant docs. Run retrieval. Count overlap.
MRR (Mean Reciprocal Rank)	Average of 1/rank of first relevant document	> 0.65	Same 30 test queries. For each, find rank of first gold-standard doc.
Context relevance	Fraction of tokens in final compressed context that are relevant to the query	> 0.70	Sample 20 agent runs. Manually rate each context chunk: relevant/irrelevant. Average fraction.
Retrieval latency	Full pipeline wall-clock time (rewrite → retrieve → rerank → compress)	< 2,000ms	time.perf_counter() around full pipeline for 10 tickers. Average and p95.
Cross-encoder lift	Improvement in Recall@10 from reranking vs. pre-rerank	> 0.10 lift	Measure Recall@10 before and after cross-encoder step on same test set.

9.2 Agent Quality Metrics

Metric	Definition	Target	Measurement method
Output schema validity	Fraction of runs producing valid Pydantic-parseable JSON on first attempt	> 0.95	Automated: track parse success/failure in logs per agent per run.
Context grounding rate	Fraction of agent's stated facts that appear in its retrieved context (not hallucinated)	> 0.85	Sample 10 outputs per agent. For each factual claim, check if it appears in the context package.
Personality consistency	Does the agent's tone, bias, and reasoning style match its defined personality?	Qualitative pass	Human review: rate 5 outputs per agent on 5-point scale for personality adherence.
Debate trigger rate	Fraction of tickers where significant agent disagreement triggers a debate round	Target 20–40%	Count debate triggers in logs. Too high = agents unreliable. Too low = no real disagreement.

Metric	Definition	Target	Measurement method
VERA veto rate	Fraction of PROCEED recommendations that VERA vetoes	Target 10–25%	Count veto=true in VERA outputs. Too low = VERA not doing its job. Too high = too conservative.

9.3 System-Level (Trading) Metrics

Metric	Target (30-day paper)	Target (90-day paper)	Phase 2 gate (go live)
Sharpe ratio	> 0.6	> 1.0	> 1.0 sustained over 90 days
Max drawdown	< 20%	< 15%	< 12% over any rolling 30-day period
Win rate	> 45%	> 50%	> 50%
Avg R:R ratio	> 1.5:1	> 2.0:1	> 2.0:1
CAGR (annualised)	> 15%	> 20%	> 20%
Factor attribution clarity	> 3 of 7 params clearly predictive	All 7 params attributed	Weights converged with confidence > 0.7

10. Developer Quick Reference

10.1 Full dependency map

```
Python packages (pip install):
  # Core AI / LLM
  google-generativeai groq openai
  # Memory / retrieval
  qdrant-client sentence-transformers rank-bm25 neo4j
  # Data / async
  pandas numpy scipy statsmodels yfinance aiohttp redis
  # Validation / utils
  pydantic tiktoken beautifulsoup4 feedparser lxml
  # Dashboard
  streamlit plotly psycopg2-binary

Docker services (all local, all free):
  timescale/timescaledb-ha:pg16   # port 5432
  redis:7-alpine                  # port 6379
  neo4j:5-community              # ports 7474, 7687
```

10.2 Environment variables

```
# .env file — never commit to git
GOOGLE_AI_STUDIO_KEY=...
GROQ_API_KEY=...
NVIDIA_NIM_KEY=...
QDRANT_URL=https://xxxx.qdrant.io
QDRANT_API_KEY=...
NEO4J_URI=bolt://localhost:7687
NEO4J_USER=neo4j
NEO4J_PASSWORD=hivemind
POSTGRES_DSN=postgresql://postgres:hivemind@localhost:5432/hivemind
REDIS_URL=redis://localhost:6379
```

10.3 Key file structure

```
hivemind/
  memory/
    manager.py          # MemoryManager class
    redis_store.py      # T2 episodic memory
    qdrant_store.py     # T3 semantic memory
    neo4j_store.py      # T5 graph memory
    procedures.py       # T4 procedural memory
  retrieval/
    bm25_engine.py      # BM25 text search
    dense_engine.py     # Vector similarity search
    graph_engine.py     # Neo4j Cypher retrieval
    fusion.py           # RRF fusion
    reranker.py         # Cross-encoder reranking
    compressor.py       # Context compression
    query_rewriter.py   # Query expansion
    iterative.py        # 2-round retrieval loop
  agents/
    base.py             # BaseAgent with Memory Manager wiring
    vega.py             # Macro analyst
    nysa.py             # News analyst
    quantra.py          # Quant analyst
```

```
    lexa.py              # Fundamental researcher
    sectora.py           # Sector specialist
    vera.py              # Critic agent
    apex.py              # Orchestrator
  swarm/
    runner.py            # asyncio execution DAG
    rate_limiter.py      # Semaphores + counters
    validator.py         # Pydantic schema validation
  feedback/
    post_mortem.py       # Post-trade analysis
    classifier.py        # Mistake type classification
    optimiser.py         # Monthly weight optimisation
  summarisation/
    progressive.py       # Weekly/monthly compression jobs
  tests/
    test_retrieval.py    # Recall metrics, latency tests
    test_agents.py       # Schema validation, grounding tests
    test_memory.py       # TTL, budget, tier tests
```

10.4 First 3 things to run on day 1

5. docker compose up -d (starts TimescaleDB, Redis, Neo4j)
6. python tests/test_infrastructure.py (verifies all 5 storage layers are reachable)
7. python memory/qdrant_store.py --backfill (populates T3 from existing agent_outputs)

End of HIVEMIND AI Layer Specification

7 agents · 5 memory tiers · hybrid retrieval · graph RAG · adaptive learning · ₹0 infrastructure